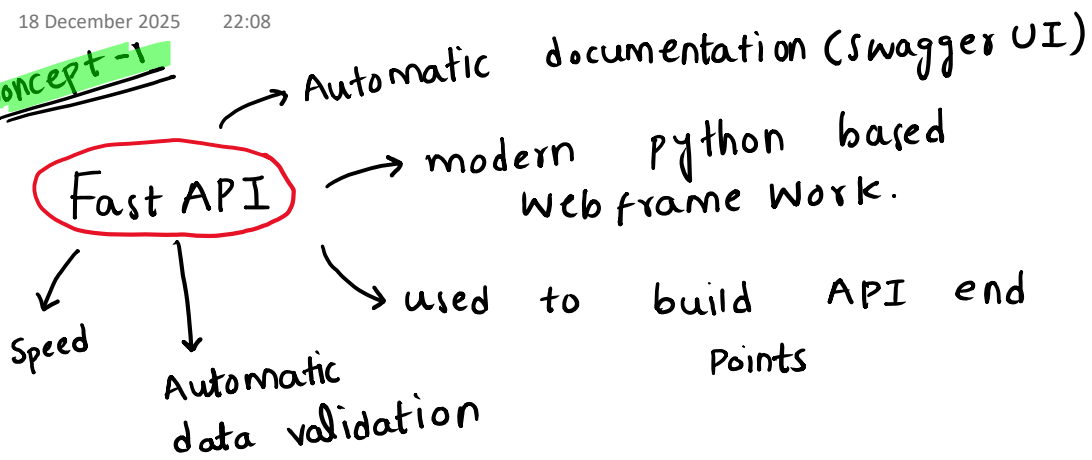
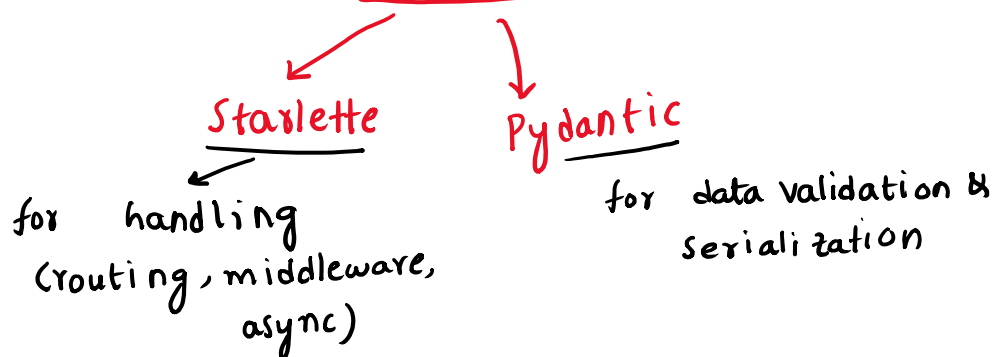


Concept-1

→ Build on top of

Concept-2

Fast API v/s Flask

Feature	Flask	FastAPI
Type hints	Optional	Core feature
Data validation	Manual	Automatic
API docs	Manual	Auto-generated
Async support	Limited	Native
Performance	Good	Excellent
Best for	Simple apps	Modern APIs, ML, microservices

Sync v/s ASync (preview)

Sync → one task at a time,
↳ server keeps all other

Sync → one task at a time,
↳ server keeps all other tasks before serving any other requests

Async → can handle many requests at a time.

concept - 3

Basic structure of FastAPI

```
main.py > ...
1  from fastapi import FastAPI
2                               class
3
4  app = FastAPI() → creating object
5  http ← → url path (home)
6  @app.get('/') → decorator
7  def greet():
8      return "Hello"
9
```

↳ automatically returns data in the JSON format.

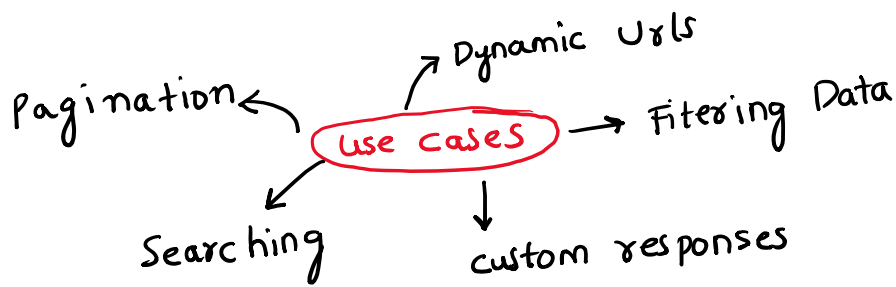
Running

`uvicorn main:app --reload`

web server (ASGI) file name object instance name ↳ reloads server for every active changes.

concept - 4

Path & Query Parameters



Path parameters → part of url's path

Query Parameters → Appear after "?" in the url

↓
can be passed through url to the backend.

```
26 users = {
27     1: 'Navaneeth',
28     2: 'Sonic',
29     3: 'Stero',
30     4: 'Avan'
31 }
32 @app.get('/users/{id}')
33 def user(id:int):
34     return users[id]
```

Dynamic value

Query Parameters

↳ Optional values, we can send value to backend to overwrite default parameters.

```
@app.get('/numbers')
def print_num(start:int=1,stop:int=10):
    a = []
    for i in range(start,stop+1):
        a.append(i)
    return a
```

→ default values are 1 & 10

the default values can be overwritten from passing through url by,

127.0.0.1:8000/numbers?start=10&stop=100

Combining both → must be passed.
 → optional value

```

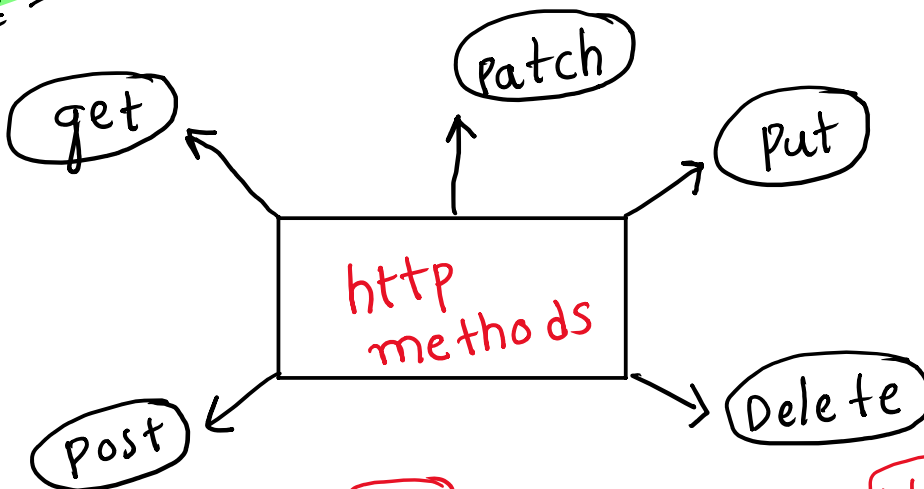
32 @app.get('/users/{id}')
33 def user(id:int, name:str=""):
34     if id not in users.keys():
35         users[id] = name
36         return f"User {name} Added Successfully"
37     else:
38         return users[id]
  
```

the parameters mentioned in the
 .get('/users/{id}')

→ must be passed
 if not we get
 Error 404 (page not found)

Where as the parameters inside function with
 default values can be skipped.

Concept-5



	when?
① Get → view users → fetch posts → read data	② Post → user registration → login → orders creation → submit forms
→ fetch/read data from server → never modify the server	→ Creates a new resource → changes server

→ never modify
the server
state.

→ changes server
state.

GET

Fetch data

No body

Cached

Visible in URL

POST

Send data

Has body

Not cached

Hidden in body

③ Put → Replace data → entire resource
↳ idempotent

↳ multiple identical requests
to a specific endpoint have
the same effect on the server's
state as a single request

→ Requires full object.

④ Patch → Partial update

↳ to update a part of resource

PUT

Full update

Replace

Requires all fields

PATCH

Partial update

Modify

Only changed fields

⑤ Delete → Remove data
↳ Deletes a resource

HTTP Method Decorator

GET

@app.get()

POST

@app.post()

PUT

@app.put()

PATCH

@app.patch()

DELETE

@app.delete()

```
#get Example
@app.get('/users')
def get_all_users():
    return users_db

@app.get('/users/{id}')
def get_user(id:int):
    return users_db.get(id)

#post
@app.post('/add')
def create_user(user:dict):
    new_id = max(users_db.keys())+1
    user['id'] = new_id
    users_db[new_id] = user
    return user
```

Handwritten red annotations:

- A red bracket groups the two GET examples, with the text *@get* written next to it.
- A red bracket groups the POST example, with the text *@Post* written next to it.

```

#put
@app.put('/users/{id}')
def replace_user(id:int, user:dict):
    user["id"] = id
    users_db[id] = user
    return user

#patch
@app.patch("/users/{id}")
def update_user_partial(id: int, updates: dict):
    users_db[id].update(updates)
    return users_db[id]

@app.delete("/users/{id}")
def delete_user(id: int):
    deleted_user = users_db.pop(id)
    return {
        "status": "deleted",
        "user": deleted_user
    }

```

Handwritten annotations in red:

- } @ put (next to the first function)
- } @ patch (next to the second function)
- } @ delete (next to the third function)

Note: FastAPI uses both [PATH+HTTP] method as a unique key, so even though every route uses /users/{id} --- it uses http method to differentiate which function need to be called.

concept-6

Data Validation with Pydantic

↳ Already covered

Dependency Injection in FastAPI

Dependency

↳ A function whose result is automatically created, managed and returned by

... automatically created, managed ...
injected into another function by

FastAPI.

- Controls Execution Order
- shares request context
- handles setup & cleanup
- Centralize cross-cutting logic
- easy testing & overrides

Manual	v/s	Dependency
We take care of ↳ calling ↳ error handling ↳ Testing ↳ cleanup		FAST API HANDLES EVERYTHING

1 Separation of concerns

Routes handle HTTP, dependencies handle logic

2 Reusability (safe reuse)

Same dependency across many routes

3 Testability

Swap real logic with fake logic easily

4 Maintainability

Change logic in one place

5 Lifecycle management

Setup + cleanup (`yield`)

Main goals of
Dependencies

Where to use dependency?

↳ if the code:

- is repeated
- is cross-cutting
(auth, db, logging)
- Needs request context
- Needs cleanup

Examples

```

# 1 DEPENDENCY: Request-based logging (needs request context)
# -----
def log_request(request: Request):
    print(f"Incoming request: {request.method} {request.url.path}")

# -----
# 2 DEPENDENCY: Extract token from request (request context)
# -----
def get_token(request: Request):
    token = request.headers.get("Authorization")
    if not token:
        raise HTTPException(status_code=401, detail="Token missing")
    return token

# -----
# 3 DEPENDENCY: Get current user (depends on token)
# -----
def get_current_user(token: str = Depends(get_token)):
    # Fake token validation
    if token != "admin-token":
        return {"username": "user", "role": "user"}
    return {"username": "admin", "role": "admin"}

# -----
# 4 DEPENDENCY: Permission check (depends on user)
# -----
def require_admin(user = Depends(get_current_user)):
    if user["role"] != "admin":
        raise HTTPException(status_code=403, detail="Admin access required")
    return True

# -----
# 5 DEPENDENCY: Pagination (query params)
# -----
def pagination(skip: int = 0, limit: int = 10):
    return {"skip": skip, "limit": limit}

# -----
# 6 DEPENDENCY: Database session (setup + cleanup)
# -----
def get_db():
    print("DB CONNECTED")
    db = {"connection": "open"}
    try:
        yield db
    finally:
        print("DB CLOSED")

```

} base functions

Depends on get-token

depends on get-current-user

} base function

} base function

```

# -----
# ROUTES
# -----
@app.get("/")
def home():
    return {"message": "Welcome to the API"}

@app.get("/public")
def public_route(log = Depends(log_request)):
    return {"message": "This is a public endpoint"}

@app.get("/profile")
def profile(
    user = Depends(get_current_user),
    log = Depends(log_request)
):
    return {
        "username": user["username"],
        "role": user["role"]
    }

@app.get("/items")
def list_items(
    paging = Depends(pagination),
    db = Depends(get_db),
    log = Depends(log_request)
):
    return {
        "pagination": paging,
        "db_status": db
    }

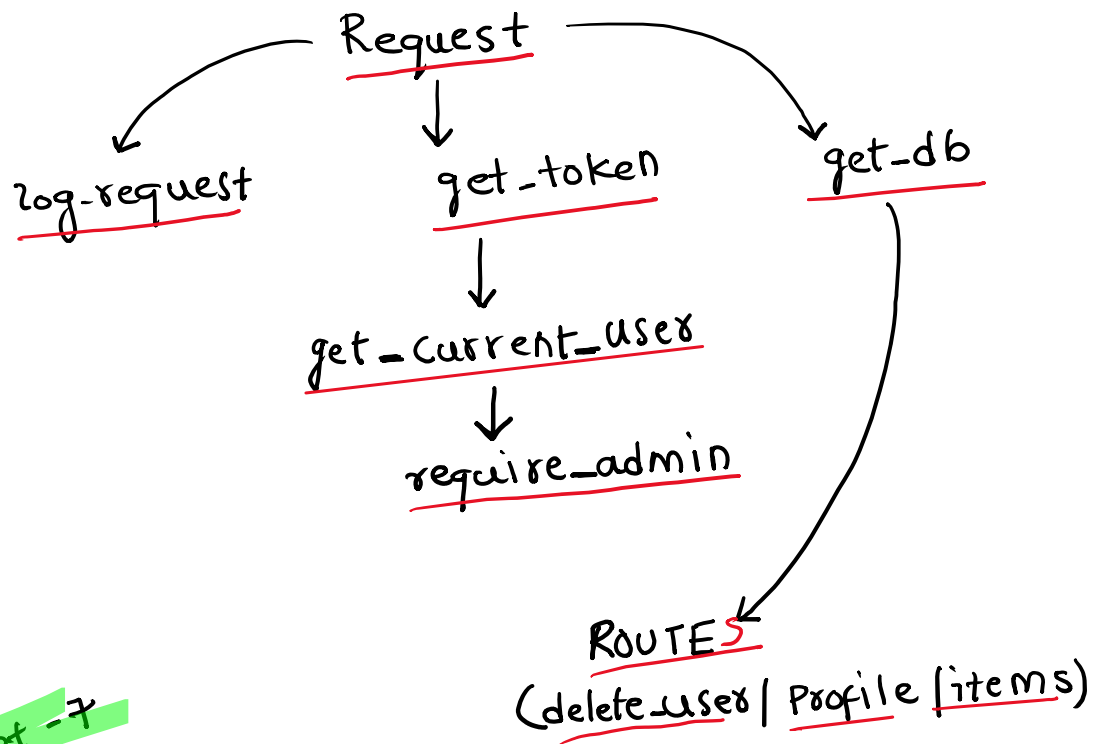
@app.delete("/admin/delete-user/{user_id}")
def delete_user(
    user_id: int,
    admin = Depends(require_admin),
    db = Depends(get_db),
    log = Depends(log_request)
):
    return {
        "status": "deleted",
        "user_id": user_id
    }

```

} home page
 } depends on log-request
 } depends on get-current-user
 } depends on log-request
 } depends on pagination, get-db, log-request
 } depends on require-admin, get-db, log-request
 → get-token depends on

So basically fastapi itself creates a dependency process life cycle that helps us to easily reuse our code in stable & efficient way.

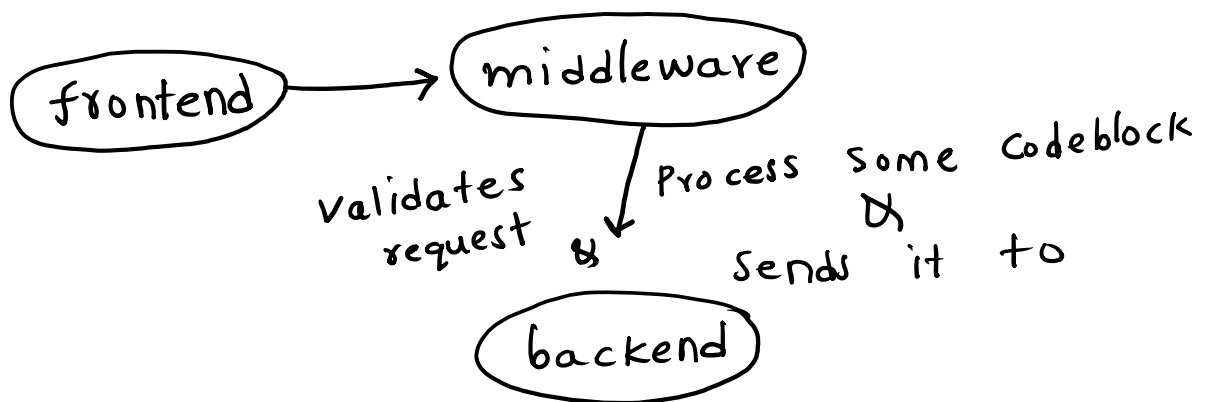
Request



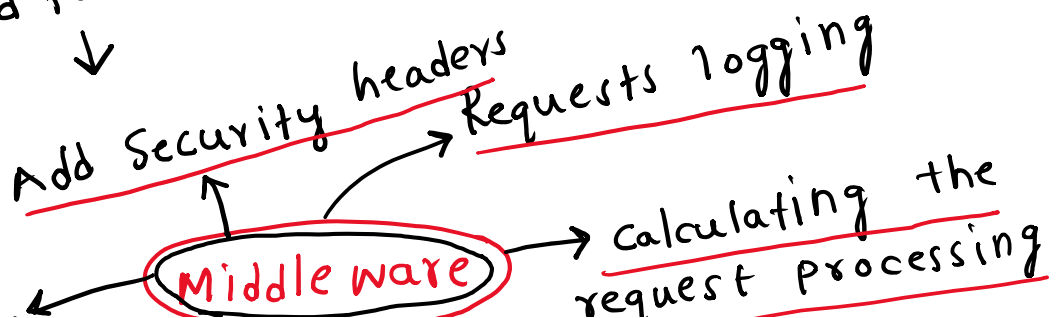
Concept - 7

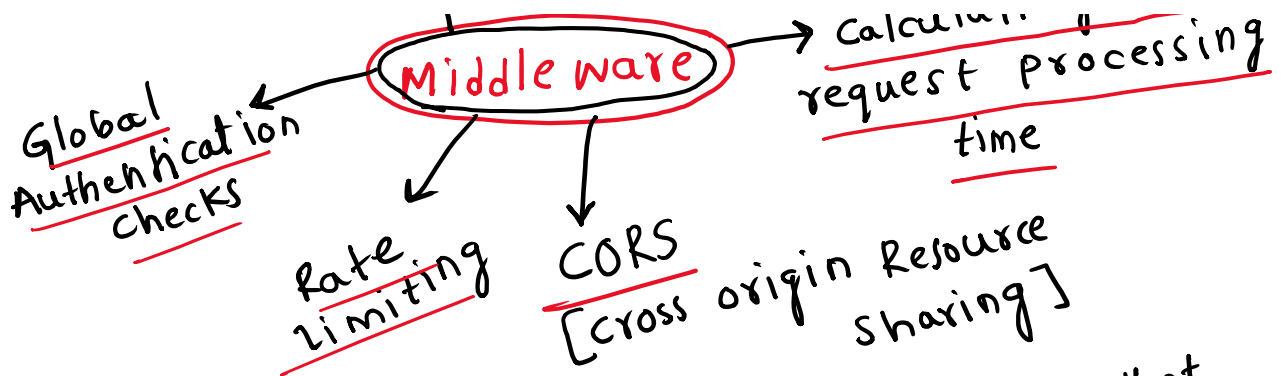
→ Middleware:

↳ a block of code, that runs for every request before calling the specified route & after the response leaves your route.



middleware used for
↓





**MUST
BE
ASYNC**

↓
browser security rule that controls whether a website is allowed to request data from another website (origin)

Define once use it everywhere

Example

```

from fastapi import FastAPI, Request
import time

app = FastAPI()

@app.middleware("http")
async def custom_middleware(request: Request, call_next):
    # BEFORE request reaches route
    print(f"Incoming request: {request.method} {request.url.path}")

    start_time = time.time()

    # Pass request to route
    response = await call_next(request)

    # AFTER route returns response
    process_time = time.time() - start_time
    print(f"Completed in {process_time:.4f} seconds")

    # Add custom header to response
    response.headers["X-Process-Time"] = str(process_time)

    return response

@app.get("/hello")
async def root():
    return {"message": "Hello World"}
  
```

→ tells server to run this middleware for every http request

→ every middleware contains two parameters.

① Request
↳ contains everything about incoming request

② call_next

↓
function that passes the request

1. Finds matching route
2. Runs dependencies
3. Executes route function
4. Returns response

function.
Passes the request
to next step.

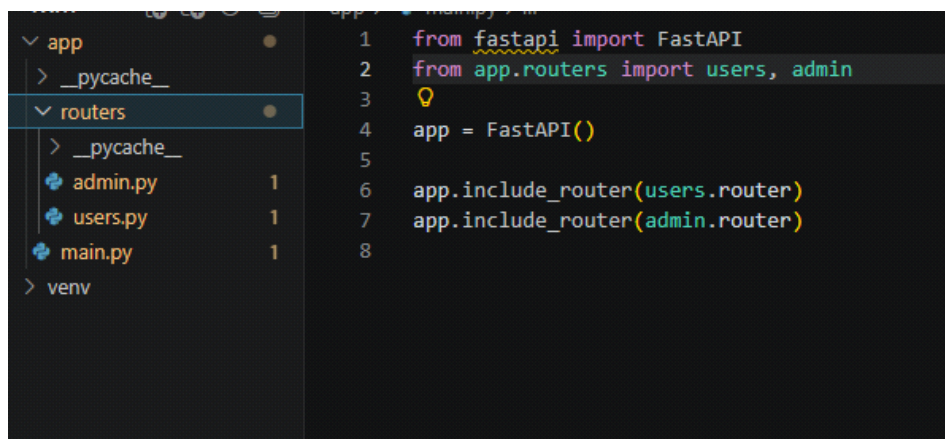
concept-8

Routers

→ a way to group all related endpoints and split your application into multiple files based on features

without routers → single file (Large APP)
↳ 200+ routes

↓
hard to maintain



The screenshot shows a code editor with a project structure on the left and code on the right. The project structure includes a folder named 'app' containing a subfolder 'routers' and files 'admin.py', 'users.py', and 'main.py'. The 'routers' folder is expanded, showing its contents. The code on the right is a Python file (likely 'main.py') that imports FastAPI and includes routers from the 'app.routers' module.

```
1 from fastapi import FastAPI
2 from app.routers import users, admin
3
4 app = FastAPI()
5
6 app.include_router(users.router)
7 app.include_router(admin.router)
8
```

In the production we use

```

app/
├── main.py
├── routers/
│   ├── users.py
│   ├── auth.py
│   ├── products.py
│   └── orders.py
├── dependencies/
│   ├── auth.py
│   └── db.py
├── models/           # DB models
├── schemas/          # Pydantic models
└── services/         # business logic

```

class fastapi.APIRouter

```

APIRouter(
    *,
    prefix="",
    tags=None,
    dependencies=None,
    default_response_class=Default(JSONResponse),
    responses=None,
    callbacks=None,
    routes=None,
    redirect_slashes=True,
    default=None,
    dependency_overrides_provider=None,
    route_class=APIRoute,
    on_startup=None,
    on_shutdown=None,
    lifespan=None,
    deprecated=None,
    include_in_schema=True,
    generate_unique_id_function=Default(generate_unique_id),
    strict_content_type=Default(True)
)

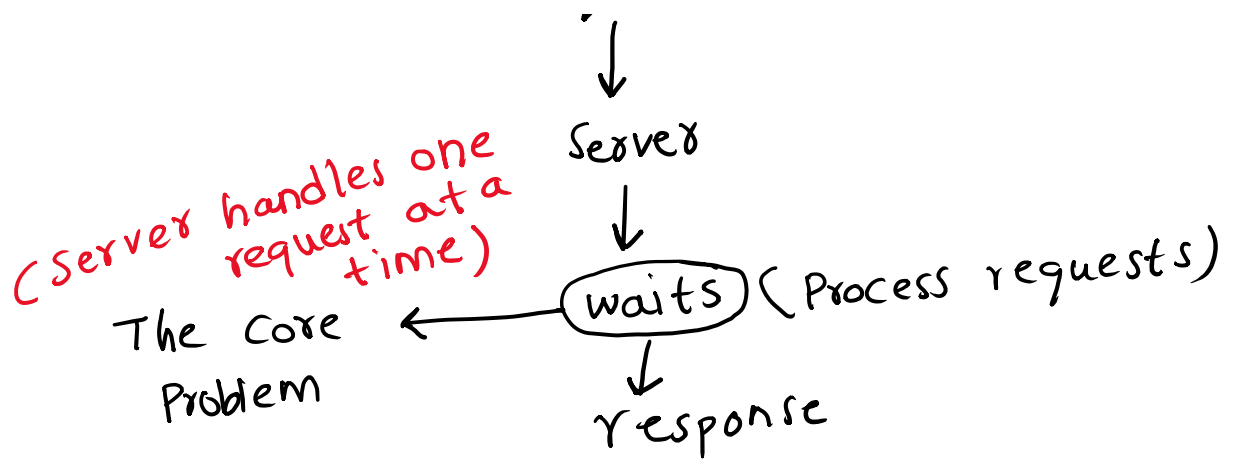
```

Concept-9

Async Programming

↳ Don't Block while waiting

When a request hits a server,
request
|



→ Say for each request the server need wait 2 seconds, if 1000 users
↳ 2000 sec (Fahh...)

So we use

ASYNC

↳ explicitly tells the server to do something while waiting
(So this allows server to serve in parallel)

→ so insted of 2000 sec, all requests can be executed in parallel and Complets in 2sec...

Async → function can pause and resume

Await → pause here, but don't block everything.

... → thread pool

Sync functions → thread pool
 Async functions → event pool

```
from fastapi import FastAPI
import time
import asyncio

app = FastAPI()

# -----
# SYNC ENDPOINT (blocking)
# -----
@app.get("/sync")
def sync_route():
    print("SYNC start")
    time.sleep(5) # blocks server
    print("SYNC end")
    return {"message": "Sync done"}

# -----
# ASYNC ENDPOINT (non-blocking)
# -----
@app.get("/async")
async def async_route():
    print("ASYNC start")
    await asyncio.sleep(5) # non-blocking
    print("ASYNC end")
    return {"message": "Async done"}
```

Scenario	Use async def	Use await	Why
Simple return (no waiting)	✗ Optional	✗	No waiting involved
Calling external API	✓	✓	Network request (I/O)
Database query (async DB)	✓	✓	DB call is I/O
File read/write (async)	✓	✓	Disk I/O
Sleep/delay (non-blocking)	✓	✓	Allows concurrency
CPU-heavy task (ML, loops)	✗	✗	No waiting, just compute
Using blocking code (time.sleep)	✗	✗	Blocks server
BackgroundTasks	✗	✗	Runs after response
Middleware	✓	✓ (call_next)	Needs async flow
Dependency (DB/API)	✓	✓	Reusable async logic

Concept - 10

Database Integration in FastAPI

for the Database Integrations and performing DB operations we use

ORM

↙
object Relational
Mapper

the ORM

↳ Converts the python objects
into the DB tables

here for ORM, we have different
options, but majorly,

→ SQLAlchemy

→ SQLModel

for connecting we need to

① Install dependencies of ORMs

② Create a db connection

↳ local db (or) an URL

③ Create Base & Model

- ④ Create tables
- ⑤ Create a DB session
- ⑥ Perform CRUD operations